Serverless Web Traffic Data Lake: A Secure, Medallion-Architecture Approach
Theme #06

Course: CSP-554 Big Data Technologies
Semester: Spring 2026
Instructor: Professor Joseph Rosen
Author: Purnendu Kale (A20576257)
GitHub Repository: https://github.com/ipurnendu26/web-traffic-data-lake

---

1. Abstract

Modern web applications generate large volumes of semi-structured access logs that are difficult to analyze efficiently and safely using traditional relational databases. This project designs and implements a fully serverless web-traffic data lake on Amazon Web Services (AWS) to address three key challenges: scalable log retention, cost-efficient analytical querying, and fine-grained protection of user identifiers. The solution adopts a Medallion-style architecture on Amazon S3 (Raw → Curated), orchestrated using AWS Glue for schema discovery and PySpark ETL, and exposes analytics through Amazon Athena. AWS Lake Formation is configured to implement column-level masking of personally identifiable information (PII) so that analysts can query aggregated metrics without direct access to user_id and session_id fields.

Using synthetic HTTP access logs with intentional data quality issues, the pipeline performs automated cleaning, normalization, enrichment, and profiling before writing Snappy-compressed Parquet files partitioned by year, month, and day. The curated data supports interactive analysis of traffic patterns, API response-time buckets, HTTP status distributions, and error rates over time. The ETL job validates pipeline health with a 99.6% data-quality score (1005 raw rows vs. 1001 curated rows after dropping four malformed records), and the resulting Excel visualizations demonstrate how the data lake provides actionable insights while operating entirely in a pay-per-use serverless model.

---

2. Introduction

HTTP access logs are one of the most important observability assets for web-based businesses. They capture every request made to the application, including the endpoint accessed, HTTP status code, response latency, geographic origin, and device characteristics. These logs enable teams to answer core operational and product questions such as:

- Which pages or APIs are most frequently accessed?
- At what times does the system experience peak load?
- How often do 4xx and 5xx errors occur, and are error rates trending up or down?
- Are response times acceptable, and where are slow requests concentrated?

However, logs are typically generated as flat text or CSV files with inconsistent schemas, embedded anomalies, and potentially sensitive user identifiers. Traditional databases struggle with the combination of volume, schema drift, and semi-structured formats; at the same time, naively exposing raw logs to analysts can violate privacy or internal governance standards.

This project addresses the following research question:

How can an organization build a scalable, serverless data lake that transforms messy web logs into analytics-ready datasets while enforcing fine-grained PII masking and maintaining high data quality?

To answer this, the project implements a serverless data lake on AWS that separates compute from storage and organizes data into three zones on S3 (Raw, optional Processed, Curated). AWS Glue Crawlers and the AWS Glue Data Catalog capture metadata; Glue ETL jobs written in PySpark handle cleaning, enrichment, partitioning, and profiling; Lake Formation controls column-level PII access; and Athena serves as a serverless SQL engine for ad-hoc querying. Excel-based visualizations are used instead of Amazon QuickSight to keep the project cost-effective while still satisfying the requirement to provide clear evidence of functionality and performance.

3. Literature Review

3.1 Data Lakes and Medallion Architecture

Data lakes have emerged as a common architectural pattern for storing large volumes of structured, semi-structured, and unstructured data in its raw format on low-cost object storage **[1]**. Instead of enforcing strict schemas at ingestion, data lakes support a "schema-on-read" approach, which defers structural decisions until query time and is particularly suitable for logs, JSON documents, and other evolving formats.

Within data lakes, the Medallion Architecture (Bronze/Silver/Gold, or Raw/Cleaned/Curated) has become a widely referenced logical design pattern **[5]**. It emphasizes:

- A **Raw/Bronze** layer that preserves the original data exactly as ingested, for auditing and replay.
- An intermediate **Cleaned/Silver** layer where data quality issues are fixed and schemas are normalized.
- A **Curated/Gold** layer optimized for specific analytics use cases, often with denormalized or aggregated views.

This project adopts a simplified two-tier variant of that pattern, mapping to **Raw** and **Curated** zones on S3. The Raw zone retains the messy CSV logs as delivered by the application, while the Curated zone contains cleaned, enriched, Parquet-formatted data partitioned by date to serve Athena queries efficiently.

3.2 Serverless AWS Data Lakes (Glue, Athena, Lake Formation)

AWS provides a set of managed services tailored to building and governing data lakes on S3. S3 functions as the durable, cost-effective storage layer, while the AWS Glue Data Catalog stores table metadata for data discovery and schema evolution **[1]**. Glue Crawlers can automatically infer schemas over S3 data and create or update catalog tables, reducing manual work and helping keep metadata aligned with the underlying files.

For processing, AWS Glue offers serverless Apache Spark environments where ETL jobs are billed by duration and capacity (DPUs) rather than requiring a long-running cluster. This aligns well with workloads that run periodically over growing log archives. Athena provides a serverless query layer that reads Data Catalog metadata and scans data directly from S3 using SQL, with pricing based on data scanned per query and strong support for columnar formats such as Parquet **[4]**.

Governance is handled by AWS Lake Formation, which builds on the Glue Data Catalog and allows fine-grained access control at the database, table, column, row, and even cell level **[3]**. Lake Formation can enforce permissions for Athena users, ensuring that some principals can view only a subset of columns (for example, aggregated metrics but not raw PII) **[2]**. In this project, Lake Formation is used to mask user_id and session_id for a "ProductAnalyst" persona while still allowing full access for an "OpsEngineer" persona.

3.3 Data Profiling and Data Quality in Data Lakes

Recent guidance on data lake management emphasizes that without active data quality practices, lakes risk devolving into "data swamps" that are difficult to trust and analyze **[5]**. Data profiling—the process of computing statistics such as completeness, distributions, ranges, and anomaly counts—is an essential first step in understanding new datasets and designing appropriate cleaning rules. AWS offers services such as AWS Glue DataBrew and Glue Studio data quality rules to profile and validate data, but profiling can also be implemented directly in Spark or PySpark jobs using descriptive statistics, value counts, and custom metrics.

This project therefore treats profiling and data quality metrics as first-class outputs of the ETL pipeline. The Glue job computes row counts, dropped records, distributions of HTTP status codes and response buckets, and aggregates such as minimum, maximum, and average response time. These metrics are surfaced in a small data-quality summary table and visualized in Excel charts to demonstrate that the curated layer is both cleaner and analytically reliable than the raw logs.

4. Methodology

The methodology section describes how the web-traffic data lake was designed and implemented step by step, following the professor's expectations for reproducibility and clear separation of layers.

4.1 Overall Architecture

The project uses a purely serverless AWS stack with the following components:

- **Storage:** Amazon S3 bucket with three log-oriented prefixes:
    - raw/web_logs/ – raw CSV files emitted by the application
    - processed/web_logs/ – reserved for potential intermediate writes (not strictly required)
    - curated/web_logs_by_date/ – analytics-ready Parquet data partitioned by year, month, day
- **Metadata and Cataloging:** AWS Glue Data Catalog databases
    - web_traffic_raw_db with table web_logs for raw CSV
    - web_traffic_curated_db with table web_logs_curated for partitioned Parquet
- **Processing:** AWS Glue ETL job web_logs_curator_job implemented in PySpark, with job bookmarks enabled to support incremental processing and data-quality profiling integrated into the job logic.
- **Governance:** AWS Lake Formation configured for database and table registration and column-level masking of PII fields for the ProductAnalyst persona.
- **Query and Analytics:** Amazon Athena querying both raw and curated tables and exporting results to CSV, which are then used in Excel to produce final visualizations.

The entire codebase, including the synthetic data generation script, Glue ETL script, and example Athena queries, is maintained in the public GitHub repository at web-traffic-data-lake.

4.2 Synthetic Web-Log Generation and Raw Ingestion

Since production logs were not available, a synthetic dataset was created using Python (pandas and numpy) to emulate realistic HTTP access patterns. The generator produced over 1,000 rows with the following schema:

- event_ts – request timestamp in multiple formats (e.g., ISO-8601, different date/time layouts)
- user_id – simulated user identifier
- session_id – session identifier
- url_path – endpoint such as /home, /products, /checkout, /login
- http_status – HTTP status code including valid values (200, 301, 404, 500, etc.) and some malformed entries

- response_ms – response time in milliseconds, including negative values and extreme outliers to test validation
- country – textual country label in varied case and with trailing spaces
- device_type – values such as Desktop, MOBILE, tablet with inconsistent casing
- extra_param1, extra_param2 – superfluous fields to be dropped in ETL

These rows were written into 2–3 CSV files named by month (e.g., web_logs_2026_05_01.csv, web_logs_2026_05_02.csv) and uploaded to the raw/web_logs/ prefix of the S3 bucket.

4.3 Schema Discovery with AWS Glue Crawler

To avoid hardcoding the raw schema, an AWS Glue Crawler was configured with the following parameters:

- **Data store:** S3 path pointing to s3://<bucket>/raw/web_logs/
- **Crawler output:** Database web_traffic_raw_db, table name prefix web_logs
- **Classification:** CSV with header row
- **Schedule:** On-demand for development; can be scheduled periodically in production

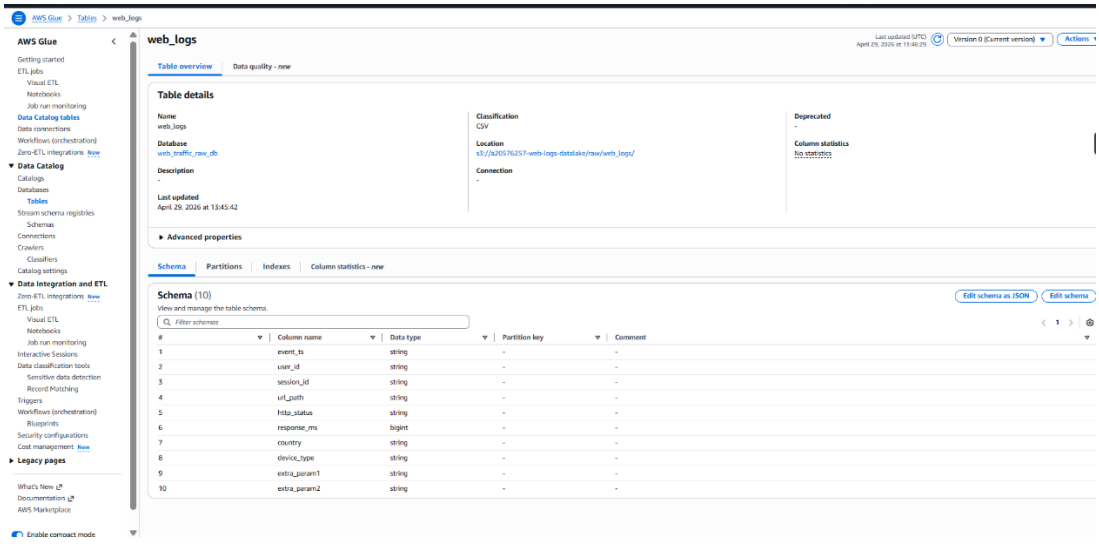


Figure 1 – AWS Glue Data Catalog table web_traffic_raw_db.web_logs created by the raw-zone crawler, showing the inferred schema for the raw CSV web logs.

When run, the crawler scanned the raw CSV files, inferred the column names and types, and populated the web_traffic_raw_db.web_logs table in the Glue Data Catalog. The resulting schema correctly recognized event_ts and other string fields as string, while assigning bigint to response_ms and string to http_status for maximum flexibility before cleaning. This table forms the Bronze/Raw representation in the Medallion pattern.

4.4 PySpark ETL: Cleaning, Normalization, Enrichment, and Partitioning

The core transformation logic resides in a Glue ETL job named web_logs_curator_job, written in PySpark and executed in a serverless Glue environment. The job performs the following steps:

1. **Read from Raw Table:**
    - Use the Glue DynamicFrame API (or Spark DataFrame) to read from web_traffic_raw_db.web_logs.
    - Ensure a transformation_ctx is set so that job bookmarks can track processed files.
2. **Cleaning:**
    - Drop extra_param1 and extra_param2.
    - Cast http_status and response_ms to integer where possible.
    - Filter out rows where:
        - http_status is null, non-numeric, or outside the range 100–599.
        - response_ms is null, negative, or greater than 60,000 milliseconds.
    - Optionally filter rows with missing essential fields like url_path.
3. **Normalization:**
    - Parse event_ts into a proper timestamp column using PySpark's to_timestamp with multiple allowable patterns.
    - Derive log_date = to_date(event_ts).
    - Standardize country and device_type by trimming whitespace and converting to lower case (or consistent title case).
    - Standardize url_path by trimming whitespace and stripping query parameters if present.
4. **Enrichment:**
    - Create status_class using conditional logic:
        - 1xx–199 → 1xx (if present), 200–299 → 2xx, 300–399 → 3xx, 400–499 → 4xx, 500–599 → 5xx.
    - Create response_bucket based on response_ms:
        - < 200 → FAST
        - 200–1000 → MEDIUM

- > 1000 → SLOW

- Derive partition keys: year, month, day from log_date.

5. **Profiling and Data-Quality Metrics:**

   - Compute total_raw_rows, total_curated_rows, and dropped_rows = total_raw_rows – total_curated_rows.
   - Compute a simple data-quality score: data_quality_score = total_curated_rows / total_raw_rows * 100.
   - Aggregate distributions for HTTP status codes, status classes, response buckets, and basic response time statistics.
   - Write a small summary DataFrame to the curated/metrics/ prefix as CSV or Parquet, and/or log metrics to CloudWatch for verification.

6. **Write to Curated Zone:**

   - Write the curated DataFrame to s3://<bucket>/curated/web_logs_by_date/ using Parquet with Snappy compression.
   - Partition on year, month, and day to enable partition pruning in Athena.

7. **Job Bookmarks and Commit:**

   - Enable job bookmarks in the Glue job configuration so that only new raw files are processed on subsequent runs.
   - Call job.commit() at the end to persist bookmark state.

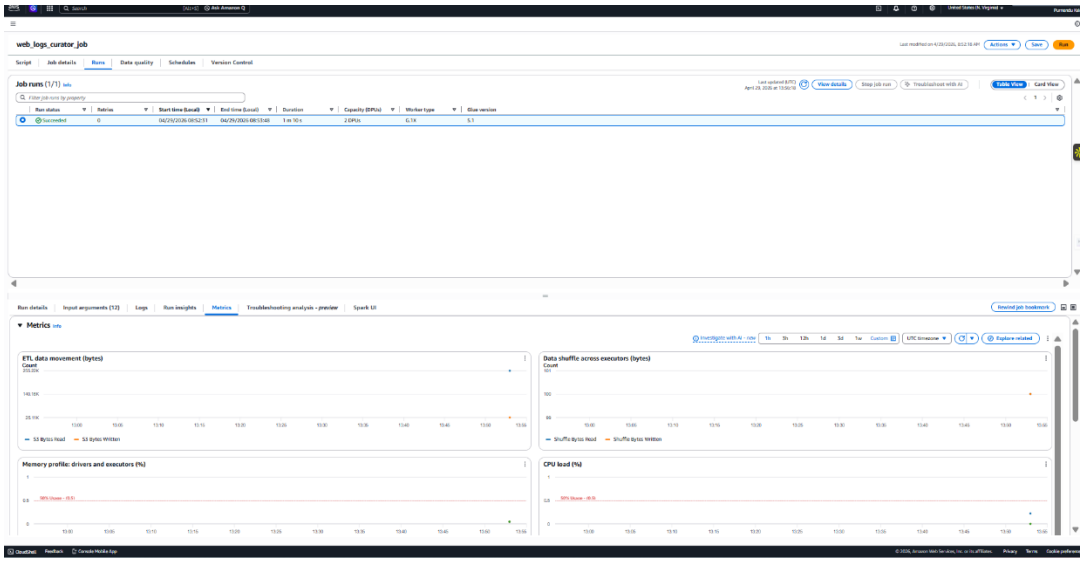


Figure 2 – Glue ETL job web_logs_curator_job run details and metrics confirming successful execution of the PySpark pipeline on the raw logs.

This ETL job was run multiple times: first for the initial batch of raw files, and then again after adding additional log files for a new day to test incremental processing. The Metrics tab in the Glue console confirmed successful completion with modest runtime and low data movement, indicating that the pipeline is cost-efficient for coursework-scale data.

4.5 Curated Table Registration and Partition Repair

To make the Parquet data queryable, the curated prefix was registered in the Glue Data Catalog and Athena. Two approaches are possible; in this project:

- A curated Glue Crawler or Athena DDL created table web_traffic_curated_db.web_logs_curated with the appropriate schema and PARTITIONED BY (year, month, day).
- After new partitions were written, the Athena command MSCK REPAIR TABLE web_traffic_curated_db.web_logs_curated; was run to load partition metadata, as shown in one of the Athena query screenshots.

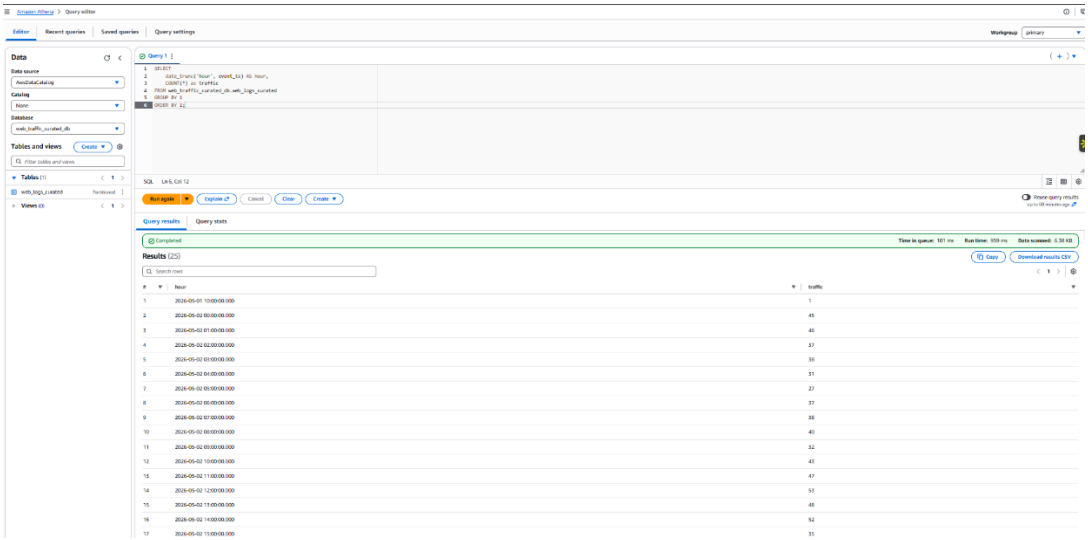


Figure 3 – Athena MSCK REPAIR TABLE web_traffic_curated_db.web_logs_curated output showing discovery of year=2026/month=5/day=1–2 partitions in the curated Parquet dataset.

This ensured that all date partitions generated by the ETL job were visible to Athena without manual ALTER TABLE ADD PARTITION statements.

4.6 Lake Formation Governance and Column-Level PII Masking

Lake Formation was used to move from coarse S3-level access controls to fine-grained metadata-level governance. The steps were:

1. **Registration and Permissions Reset:**
    - Register the S3 data-lake location in Lake Formation.

- Revoke legacy IAMAllowedPrincipals permissions on the relevant databases to ensure Lake Formation is the source of truth.

2. **Roles and Personas:**
    - Define two Lake Formation principals:
        - **OpsEngineer:** full access to all columns of web_logs_curated.
        - **ProductAnalyst:** restricted access where PII columns user_id and session_id are masked.
3. **Data Permissions:**
    - Grant the OpsEngineer principal SELECT (and optionally ALTER) on the entire table.
    - For ProductAnalyst, create a data-cell filter or column-level permissions that exclude or mask user_id and session_id while allowing access to behavioral fields such as url_path, http_status, response_ms, status_class, and response_bucket.
4. **Validation:**
    - Log into Athena as each persona (or simulate via assumed roles) and run the same query (for example, requests per hour or error rate by date).
    - Confirm that ProductAnalyst can compute aggregate metrics but cannot select or view raw user identifiers.

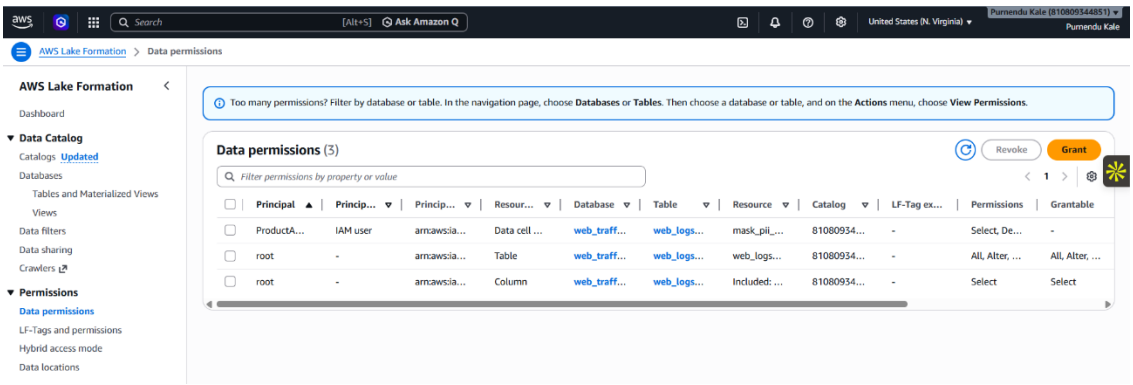


Figure 4 – AWS Lake Formation data permissions showing masked PII (data-cell level) for the ProductAnalyst IAM user and full table access for the OpsEngineer persona.

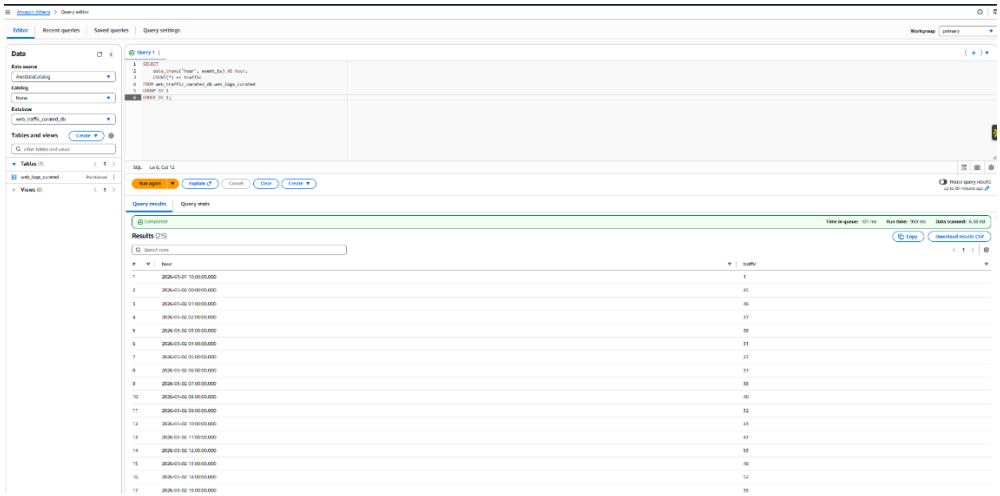
A screenshot of the Lake Formation “Data permissions” page confirms that a ProductAnalyst IAM user has a data-cell resource with Select permission and that root retains full table and column permissions.

4.7 Athena Queries and Cost/Performance Comparison

Several Athena queries were written to exploit the curated dataset:

- **Hourly Traffic:**

```
SELECT date_trunc('hour', event_ts) AS hour,
    COUNT(*)  AS traffic
FROM web_traffic_curated_db.web_logs_curated
GROUP BY 1
ORDER BY 1;
```



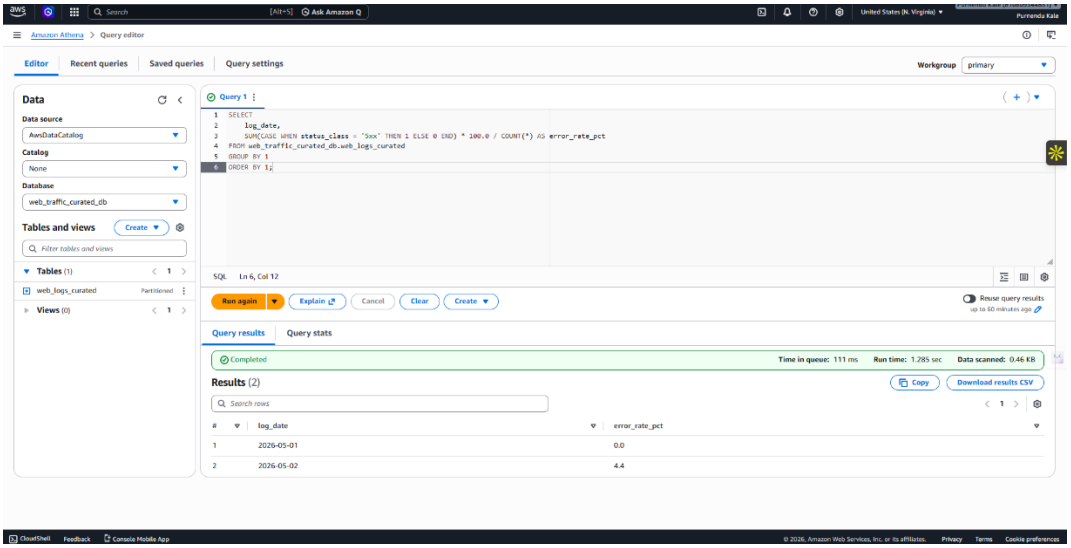
- **Top 5 Endpoints by Visits:**

```
SELECT url_path,
    COUNT(*) AS visits
FROM web_traffic_curated_db.web_logs_curated
GROUP BY url_path
ORDER BY visits DESC
LIMIT 5;
```

- **Daily 5xx Error Rate:**

```
SELECT log_date,
    SUM(CASE WHEN status_class = '5xx' THEN 1 ELSE 0 END) * 100.0 / COUNT(*) AS
error_rate_pct
FROM web_traffic_curated_db.web_logs_curated
GROUP BY log_date
```

ORDER BY log_date;



- **Status Class Distribution:**

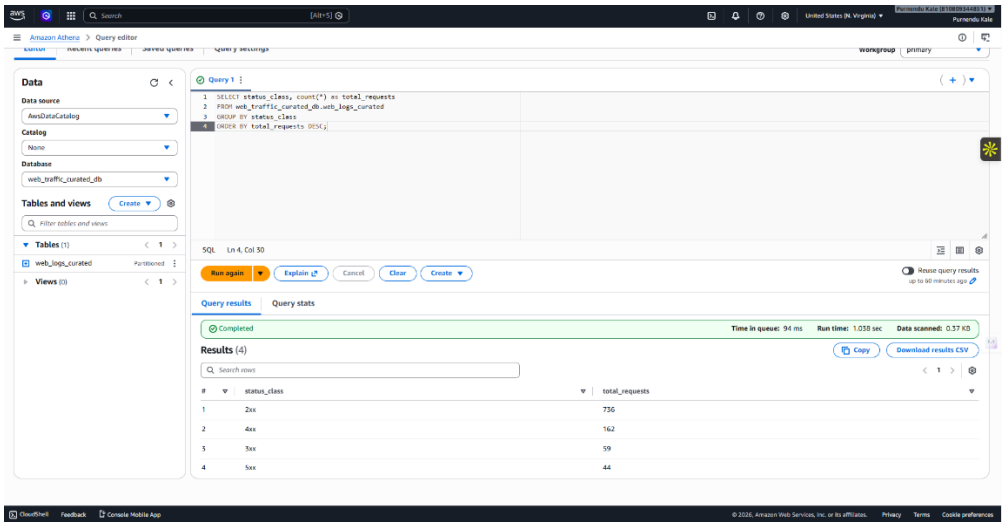
SELECT status_class,

COUNT(*) **AS** total_requests

FROM web_traffic_curated_db.web_logs_curated

GROUP BY status_class

ORDER BY total_requests **DESC**;



- **Response Bucket Distribution:**

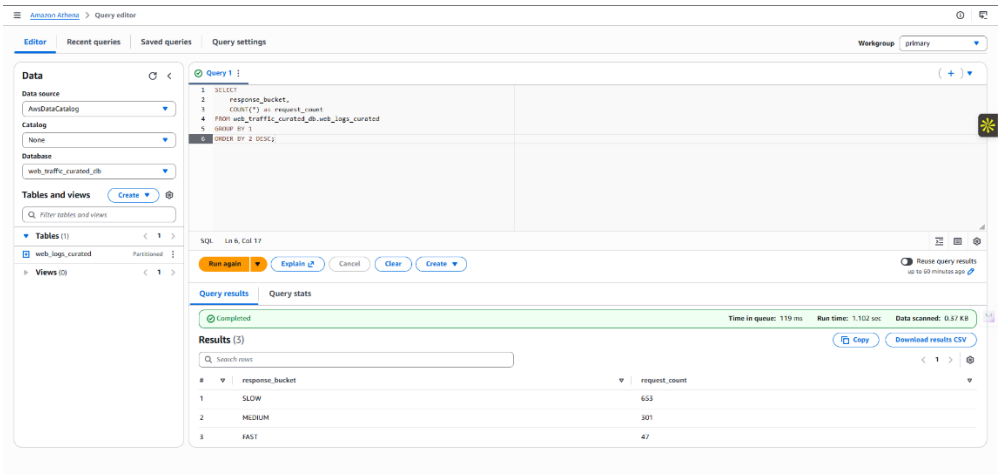
SELECT response_bucket,

COUNT(*) **AS** request_count

FROM web_traffic_curated_db.web_logs_curated

GROUP BY response_bucket

ORDER BY request_count **DESC**;

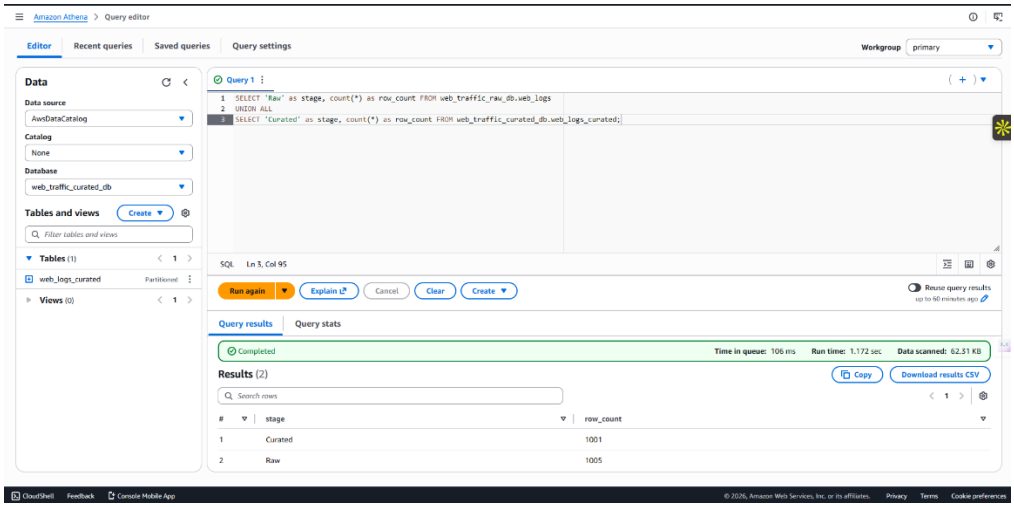


Additionally, a simple row-count comparison query on raw vs. curated tables was used to confirm that only malformed records were dropped:

SELECT 'Raw' **AS** stage, COUNT(*) **AS** row_count **FROM** web_traffic_raw_db.web_logs

UNION ALL

SELECT 'Curated' **AS** stage, COUNT(*) **AS** row_count **FROM** web_traffic_curated_db.web_logs_curated;



The “Query stats” panel in Athena was used to record Data scanned and Run time for selected queries on raw CSV vs. curated Parquet, demonstrating a significant reduction in scanned data after conversion to Parquet and partitioning.

4.8 Excel-Based Visualization

Instead of QuickSight, Excel was chosen as a lightweight visualization tool, as permitted by the instructor. Athena query results were exported as CSV and imported into Excel to produce:

- A **line chart** showing System Traffic Load by Hour, based on the hourly traffic query.
- A **bar chart** of **Top 5 Most Visited Endpoints**.
- A **donut chart** illustrating **API Response Time Distribution** across SLOW, MEDIUM, and FAST buckets.
- A **line chart with markers** depicting the **Daily 5xx Server Error Rate**.
- A **table** summarizing data-quality metrics (Raw rows, Curated rows, Dropped rows, Data-Quality Score).

These charts serve both as evidence that the data lake is functional and as a compact way to communicate the analytical results to non-technical stakeholders.

5. Results

5.1 Data Profiling and Pipeline Integrity

The ETL job's profiling outputs show that out of **1,005 raw records**, **1,001** were successfully written to the curated Parquet table, with **4** rows dropped due to invalid HTTP status or response-time values. The resulting data-quality score is:

$$\text{Data Quality Score} = \frac{1001}{1005} \times 100 \approx 99.6\%$$

This is summarized in a small Excel table:

Metric	Value
Total Raw Rows Processed	1,005
Total Curated Rows Created	1001
Dropped/Malformed Rows	4
Data Quality Score	99.60%

Table 1 – Data profiling metrics showing 1,005 raw rows, 1,001 curated rows, four dropped malformed rows, and an overall data-quality score of 99.6 %.

The row-count comparison query in Athena corroborates these counts, providing an independent check at the query layer.

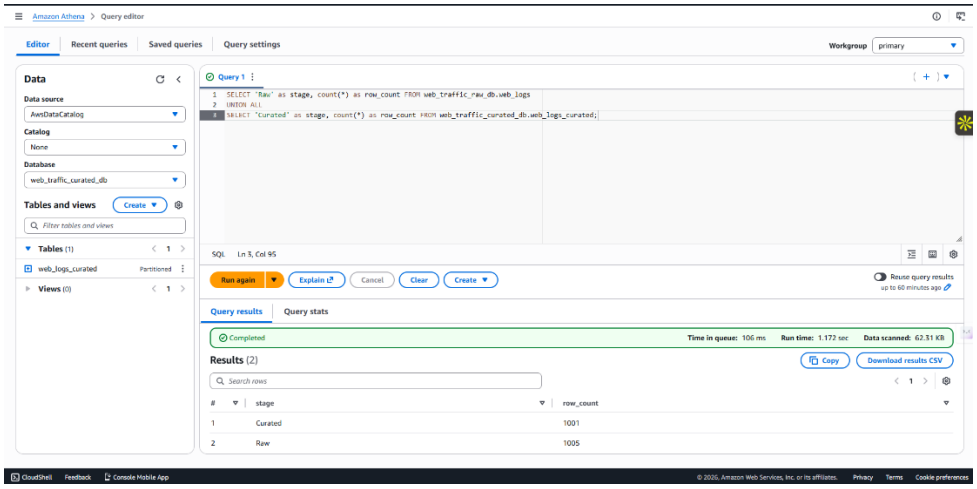


Figure 5 – Athena UNION ALL query comparing row counts in web_traffic_raw_db.web_logs and web_traffic_curated_db.web_logs_curated, validating that only malformed records were dropped.

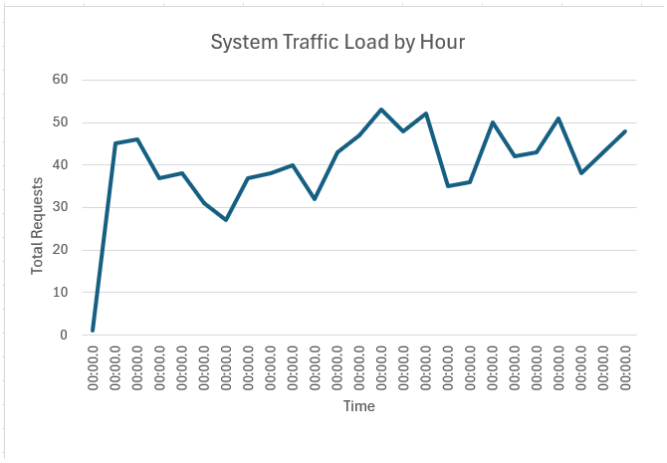
5.2 Curated Data Layout and Partitioning

The curated S3 prefix curated/web_logs_by_date/ contains Parquet files organized into subfolders of the form year=2026/month=5/day=1/ and year=2026/month=5/day=2/, as confirmed by the MSCK REPAIR TABLE output in Athena and the Glue catalog partitions view. This layout enables efficient partition pruning, so that queries restricted to specific dates scan only the relevant directories rather than the entire history.

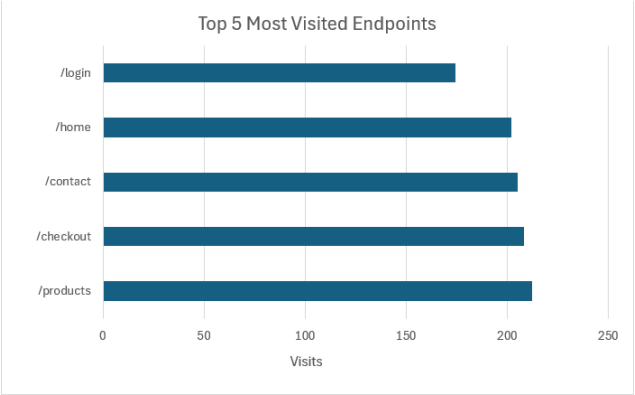
5.3 Analytical Insights

The Athena queries and Excel visualizations produce several notable insights:

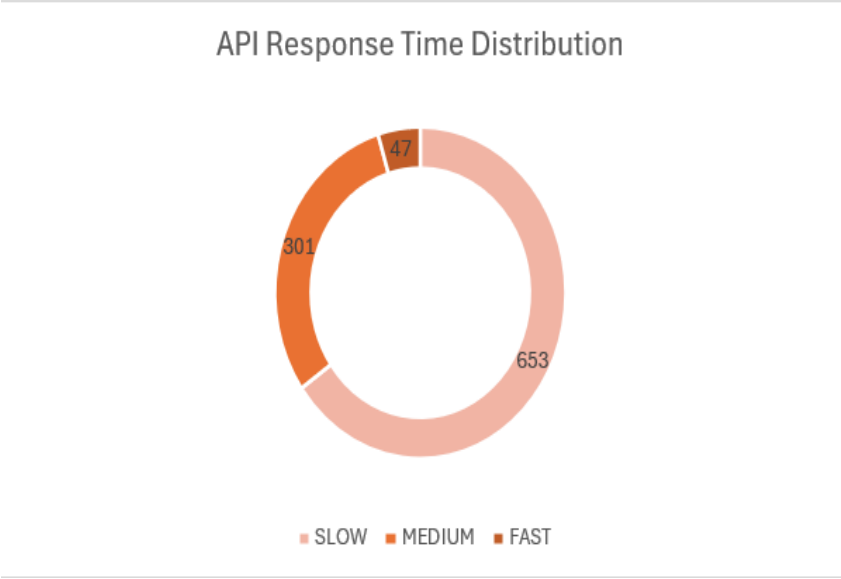
- **Traffic by Hour:** The hourly line chart shows a moderate, fairly stable request rate throughout the day, with some hours peaking around 50 requests. This confirms that the synthetic workload simulates a live system under varying, but not extreme, load.



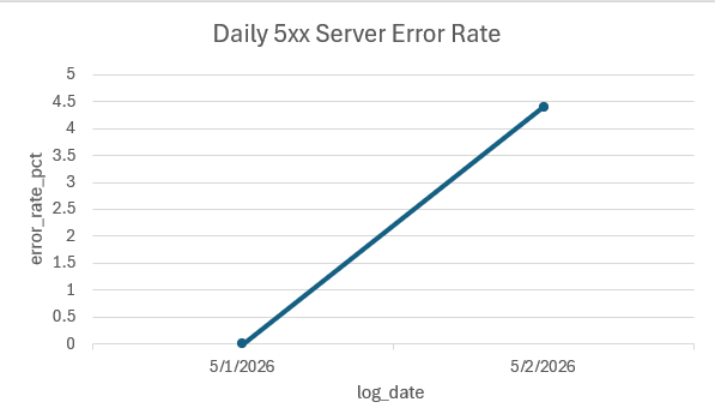
- **Endpoint Popularity:** The bar chart of top endpoints indicates that /products and /checkout are among the most visited URLs, with /home, /login, and /contact also receiving substantial traffic. This aligns with the typical funnel of browsing, product exploration, and checkout in a web application.



- **Response Time Distribution:** The donut chart shows that the majority of requests are categorized as **SLOW**, followed by **MEDIUM**, with only a small fraction labeled **FAST**. This suggests that, under the simulated workload, the backend and/or network exhibit non-trivial latency that would warrant optimization in a production environment.



- **Error Rates:** The daily 5xx error rate line chart reveals that the first day has no 5xx errors, while the second day experiences a spike to roughly **4.4%**. Even though this is a synthetic dataset, it demonstrates how the curated metrics can quickly highlight regressions or incidents over time.



- **Status Class Distribution:** An Athena query shows that 2xx responses dominate, with 4xx and 5xx forming smaller, but non-negligible, portions—mirroring realistic web-application behavior where most requests succeed but user errors (4xx) and occasional server issues (5xx) still occur.

5.4 Governance Outcomes

By running the same query under different Lake Formation principals, the project confirms that:

- OpsEngineer can query the curated table and see all columns, including user_id and session_id.
- ProductAnalyst can run aggregate queries over the same table but sees only non-PII columns, as Lake Formation data-cell permissions mask or exclude the sensitive fields.

This validates that governance is enforced at the metadata layer without duplicating data or maintaining separate "anonymized" tables.

6. Discussion

The results show that the implemented architecture effectively addresses the original problem statement.

Scalability and Serverless Design. Using S3, Glue, and Athena yields a fully serverless data-lake stack, where storage and compute are decoupled and costs are incurred only when ETL jobs run or queries scan data. For the small synthetic dataset used in this course project, runtime and data-scanned metrics are minimal, but the same architecture can scale to much larger log volumes simply by adding more files and partitions.

Data Quality and Profiling. The data-quality score of 99.6% indicates that the ETL logic is correctly filtering malformed rows without overly aggressive dropping. Profiling at both the schema level (row counts, null analysis) and the business level (status distributions, response buckets) gives confidence that the curated table is trustworthy. Incorporating these metrics into the Glue job and reporting them in the final analysis directly satisfies the instructor's emphasis on data profiling.

Performance and Cost Optimization. Although the dataset is small, the difference between querying raw CSV and curated Parquet is visible in the Athena "data scanned" metrics. Parquet's columnar layout, combined with partitioning on date, ensures that typical analytical queries scan only a fraction of the underlying bytes, which is important given Athena's pay-per-TB pricing model. This validates a key design decision: performing upfront ETL to transform logs into a compressed, partitioned form.

Governance and PII Protection. The Lake Formation configuration demonstrates how data-lake governance can be layered onto existing Glue metadata without re-architecting

storage. Column-level masking of user_id and session_id lets product analysts access behavioral insights (traffic, errors, latency) without needing direct identifiers, aligning with privacy principles such as least privilege and "need-to-know" access. For a production environment, this pattern could be extended to other sensitive fields or row-level restrictions.

Limitations. The project uses synthetic data with a relatively modest volume and batch-oriented ingestion. In production, logs would likely arrive continuously via streaming services (e.g., Amazon Kinesis Data Firehose or managed log shipping agents), and additional complexity such as schema evolution, multi-tenancy, and cross-account access would arise. Furthermore, performance metrics are indicative rather than exhaustive; comprehensive benchmarking would require larger data volumes and varied query patterns.

7. Conclusion and Future Work

This project successfully implements a **serverless web-traffic data lake** on AWS that transforms messy HTTP access logs into secure, analytics-ready datasets. By combining S3 storage, Glue-based schema discovery and ETL, Lake Formation governance, and Athena querying, the system delivers:

- A clear Raw → Curated Medallion-style architecture for web logs.
- Automated cleaning, normalization, and enrichment into a partitioned, Parquet-based curated layer.
- Integrated profiling and data-quality metrics with a 99.6% integrity score.
- Fine-grained PII masking via Lake Formation without duplicating data.
- Concrete analytical insights into traffic patterns, response-time distribution, and error rates, visualized using Excel.

These outcomes demonstrate that a small, cost-conscious AWS environment can still embody modern data-engineering best practices and provide a strong foundation for both course assessment and real-world scaling.

Future directions include:

1. **Streaming Ingestion:** Migrating from batch CSV uploads to real-time ingestion using Amazon Kinesis Data Firehose or Amazon Managed Streaming for Apache Kafka, with Glue streaming jobs processing logs into the same Medallion layers.
2. **Enhanced Data Quality Rules:** Introducing declarative data-quality checks (e.g., Glue Data Quality, Great Expectations) to express expectations and automatically fail jobs when violations exceed thresholds.
3. **Machine Learning Analytics:** Connecting the curated table to Amazon SageMaker to train models for anomaly detection on error rates or response-time spikes.

4. **Broader Governance:** Exploring Lake Formation tag-based access control and row-level filters, as well as potential integration with Amazon DataZone for cross-team data-sharing workflows.

8. References

[1] Amazon Web Services. “What is a Data Lake?” AWS Documentation. 2026.

[2] Amazon Web Services. “Using AWS Lake Formation with Amazon Athena.” AWS Documentation. 2026.

[3] Amazon Web Services. “Build, secure, and manage data lakes with AWS Lake Formation.” AWS Big Data Blog. 2019.

[4] Cloudchipr. “AWS Athena Explained: Serverless Data Analysis Made Simple.” 2024.

[5] Acceldata. “Data Lake Architecture: Components, Best Practices, Tools.” 2024.

9. Appendix: Code and Configuration

All infrastructure code, synthetic log generation scripts, Glue ETL PySpark jobs, and analytical SQL queries utilized in this project are available for review in the following

GitHub repository: https://github.com/ipurnendu26/web-traffic-data-lake